Joseph Winkie
4/10/26

Survey & Application of Regression Techniques

Technical Demonstration

Dataset overview

Tutorials

Great tutorials and walkthroughs by GeeksForGeeks - https://www.geeksforgeeks.org/machine-learning/what-is-regression-analysis/

Bike Sharing Demand

Dataset containing bikeshare usage given various environmental factors
Sourced from Kaggle - http://tiny.cc/e8o1101

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
10880	2012-12-19 17:00:00	4	0	1	1	16.4	20.455	50	26.0027	26	536	562
10881	2012-12-19 18:00:00	4	0	1	1	15.58	19.695	50	23.9994	23	546	569
10882	2012-12-19 19:00:00	4	0	1	1	15.58	19.695	50	26.0027	7	329	336
10883	2012-12-19 20:00:00	4	0	1	1	14.76	17.425	57	15.0013	10	231	241
10884	2012-12-19 21:00:00	4	0	1	1	13.94	15.91	61	15.0013	4	164	168
10885	2012-12-19 22:00:00	4	0	1	1	13.94	17.425	61	6.0032	12	117	129
10886	2012-12-19 23:00:00	4	0	1	1	13.12	16.665	66	8.9981	4	84	88

Medical Cost Personal Datasets

Dataset sourced from Machine Learning with R by Brett Lantz from a public domain data.
Retrieved from Kaggle - http://tiny.cc/n8o1101

	age	sex	bmi	children	smoker	region	charges
1332	23	female	33.4	0	no	southwest	10795.93733
1333	52	female	44.7	3	no	southwest	11411.685
1334	50	male	30.97	3	no	northwest	10600.5483
1335	18	female	31.92	0	no	northeast	2205.9808
1336	18	female	36.85	0	no	southeast	1629.8335
1337	21	female	25.8	0	no	southwest	2007.945
1338	61	female	29.07	0	yes	northwest	29141.3603

Getting Ready

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import SGDRegressor
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
```

✓ [1] 1s 36ms

```python
from sklearn.metrics import r2_score, mean_squared_error
```

✓ [2] 17ms

```python
import math
```

✓ [3] < 10 ms

Import Bikeshare Data

```python
# read in the data
bikeshare_df = pd.read_csv('bikeshare_train.csv')
bikeshare_train_df = bikeshare_df[:math.floor(len(bikeshare_df)*0.8)]
bikeshare_test_df = bikeshare_df[math.ceil(len(bikeshare_df)*0.8):]
bikeshare_kaggle_test_df = pd.read_csv('bikeshare_test.csv')
```
✓ [4] 37ms

- Dataframe from csv
- Split into train and validation sets
- Scale features

```python
# 1. Data Preparation (Bikeshare Dataset)
features_multi = ['temp', 'humidity', 'windspeed']
target = 'count'

# Scaling is MANDATORY for Gradient Descent/SGD
scaler = StandardScaler()
bikeshare_train_all_scaled = scaler.fit_transform(bikeshare_train_df[features_multi])
bikeshare_train_y = bikeshare_train_df[target].values
```
✓ [6] 65ms

```python
# 1. Scale your testing features (must use the same scaler as training!)
bikeshare_test_full_scaled = scaler.transform(bikeshare_test_df[['temp', 'humidity', 'windspeed']])
```
✓ [7] 17ms

Import Insurance Data

```python
insurance_df_raw = pd.read_csv('insurance.csv')
```
✓ [8] 10ms

```python
# Converting categories to numbers for the model
insurance_df = pd.get_dummies(insurance_df_raw, columns=['sex', 'smoker', 'region'], drop_first=True)
```
✓ [9] 12ms

```python
# split into train and test
insurance_train_df = insurance_df[:math.floor(len(insurance_df)*0.8)]
insurance_test_df = insurance_df[math.ceil(len(insurance_df)*0.8):]
```
✓ [10] 14ms

```python
# Target variable (Charges)
# Isolate the target (y) for both sets
insurance_y_train = insurance_train_df['charges'].values
insurance_y_test = insurance_test_df['charges'].values

# Create feature sets (X) by dropping the target
insurance_X_train_all = insurance_train_df.drop('charges', axis=1)
insurance_X_test_all = insurance_test_df.drop('charges', axis=1)

scaler = StandardScaler()
```
✓ [12] 31ms

```python
# Use 'age' as the single independent variable [cite: 31]
insurance_X_train_simple = scaler.fit_transform(insurance_X_train_all[['age']])

# transform for testing
insurance_X_test_simple = scaler.transform(insurance_X_test_all[['age']])
```
✓ [13] 13ms

- Dataframe from csv
- Split into train and validation sets
- Scale only age
  - Binary data has no scaling

Linear Bikeshare

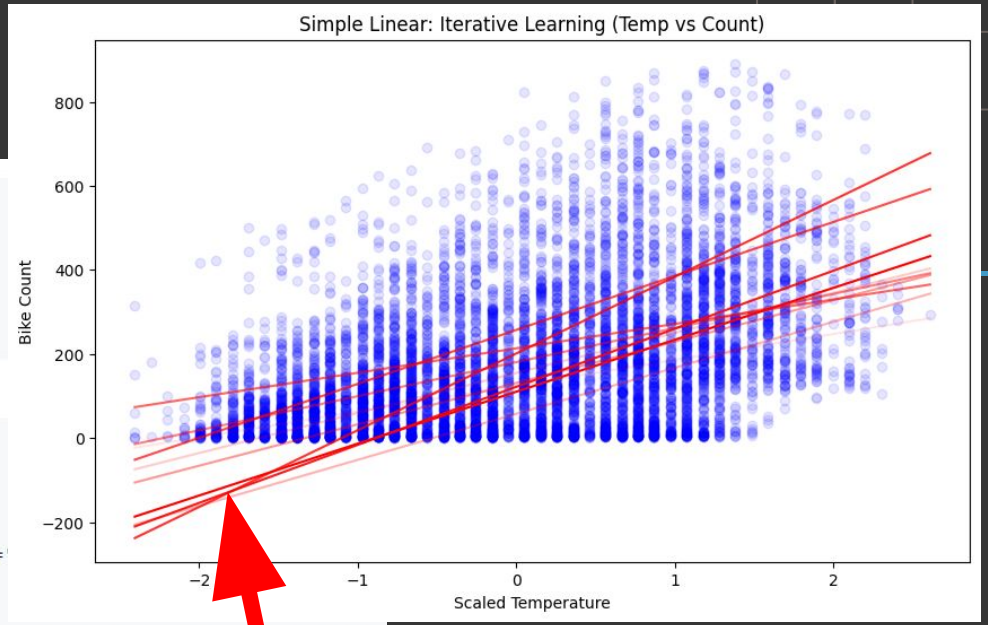
```python
chunk_count = 10
bikeshare_record_count = len(bikeshare_train_df)
bikeshare_chunk_size = bikeshare_record_count // chunk_count

print((chunk_count, bikeshare_record_count, bikeshare_chunk_size), )
```

✓ [14] 26ms

```
(10, 8708, 870)
```

```python
bikeshare_train_x_simple = bikeshare_train_all_scaled[:, [0]] # Just 'temp'
model_simple = SGDRegressor(learning_rate='constant', eta0=0.01)

plt.figure(figsize=(10, 6))
plt.scatter(bikeshare_train_x_simple, bikeshare_train_y, color='blue', alpha=0.1, label=

for i in range(0, bikeshare_record_count, bikeshare_chunk_size):
    model_simple.partial_fit(bikeshare_train_x_simple[i:i + bikeshare_chunk_size], bikeshare_train_y[i:i + bikeshare_chunk_size])

    # Plotting the intermediate line
    bikeshare_x_range = np.linspace(bikeshare_train_x_simple.min(), bikeshare_train_x_simple.max(), 100).reshape(-1, 1

    # plot the predicted line for each chunk, getting darker with each "pass"
    plt.plot(bikeshare_x_range, model_simple.predict(bikeshare_x_range), color='red', alpha=i / bikeshare_record_count)

plt.title("Simple Linear: Iterative Learning (Temp vs Count)")
plt.xlabel("Scaled Temperature")
plt.ylabel("Bike Count")
plt.show()
```

✓ [15] 190ms



Linear Bikeshare (Cont.)

```python
# Slice out only the 'temp' column (index 0) for your simple model
# This matches the [:, [0]] slicing you used during training
bikeshare_x_test_simple_scaled = bikeshare_test_full_scaled[:, [0]]

bikeshare_y_test = bikeshare_test_df['count'].values

# 2. Predict on the unseen testing data
bikeshare_y_pred = model_simple.predict(bikeshare_x_test_simple_scaled)

# 3. Calculate Key Metrics
bikeshare_simple_r2 = r2_score(bikeshare_y_test, bikeshare_y_pred)
bikeshare_simple_rmse = np.sqrt(mean_squared_error(bikeshare_y_test, bikeshare_y_pred))

print(f"R-squared: {bikeshare_simple_r2:.4f}")
print(f"RMSE: {bikeshare_simple_rmse:.2f}")
```

✓ [16] 52ms

```
R-squared: -0.3599
RMSE: 253.74
```

We did worse than just averaging all the data flat!!

Linear Insurance

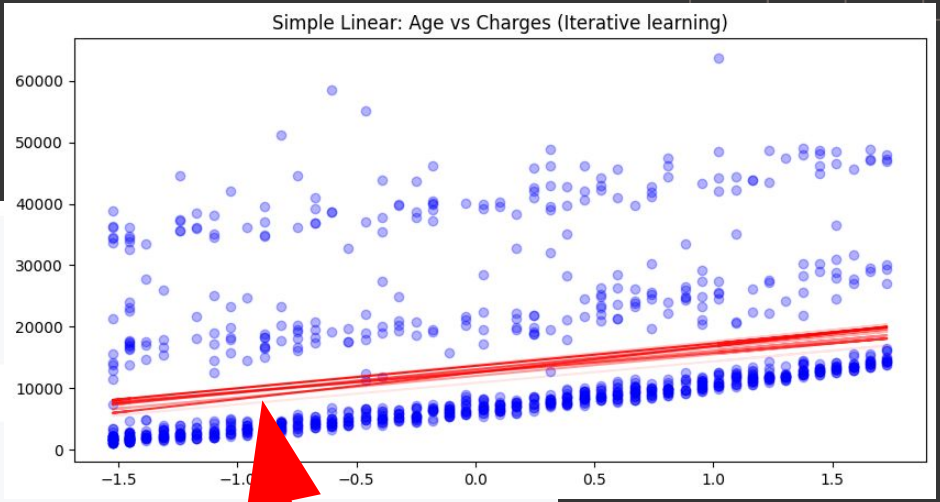
```python
insurance_train_record_count = len(insurance_X_train_simple)
insurance_train_chunk_size = insurance_train_record_count // chunk_count

print((chunk_count, insurance_train_record_count, insurance_train_chunk_size), )
```

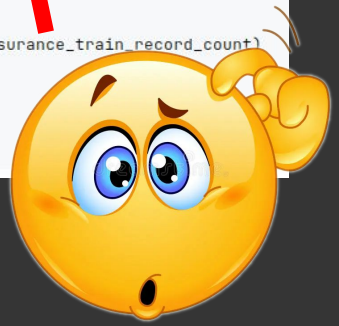
✓ [18] 58ms

```
(10, 1070, 107)
```

```python
model_simple = SGDRegressor(learning_rate='constant', eta0=0.01)

plt.figure(figsize=(10, 5))
plt.scatter(insurance_X_train_simple, insurance_y_train, alpha=0.3, color='blue', label='Actual Data')

# Showing the iterative "Gradient Descent" steps [cite: 206]
for i in range(0, insurance_train_record_count, insurance_train_chunk_size):
    model_simple.partial_fit(insurance_X_train_simple[i:i+100], insurance_y_train[i:i+100])
    plt.plot(insurance_X_train_simple, model_simple.predict(insurance_X_train_simple), color='red', alpha=i/insurance_train_record_count)

plt.title("Simple Linear: Age vs Charges (Iterative learning)")
plt.show()
```

✓ [21] 164ms





Linear Insurance (Cont.)

```python
# Simple insurance model: use only the 'age' feature

# Keep a dedicated scaler for the simple model
age_scaler = StandardScaler()

# Fit only on the training 'age' column
insurance_X_train_simple = age_scaler.fit_transform(insurance_X_train_all[['age']])

# Transform the test 'age' column using the same scaler
insurance_X_test_simple = age_scaler.transform(insurance_X_test_all[['age']])

# Train the model if needed
model_simple = SGDRegressor(learning_rate='constant', eta0=0.01)
model_simple.partial_fit(insurance_X_train_simple, insurance_y_train)

# Predict
insurance_y_pred = model_simple.predict(insurance_X_test_simple)

# Evaluate
insurance_simple_r2 = r2_score(insurance_y_test, insurance_y_pred)
insurance_simple_rmse = np.sqrt(mean_squared_error(insurance_y_test, insurance_y_pred))

print(f"R-squared: {insurance_simple_r2:.4f}")
print(f"RMSE: {insurance_simple_rmse:.2f}")
```

✓ [22] 26ms

```
R-squared: 0.0304
RMSE: 12135.56
```

- strong clustering, throws off a weighted regression model
- can't approximate all data
- while this is easy to explain, it could be dangerously wrong

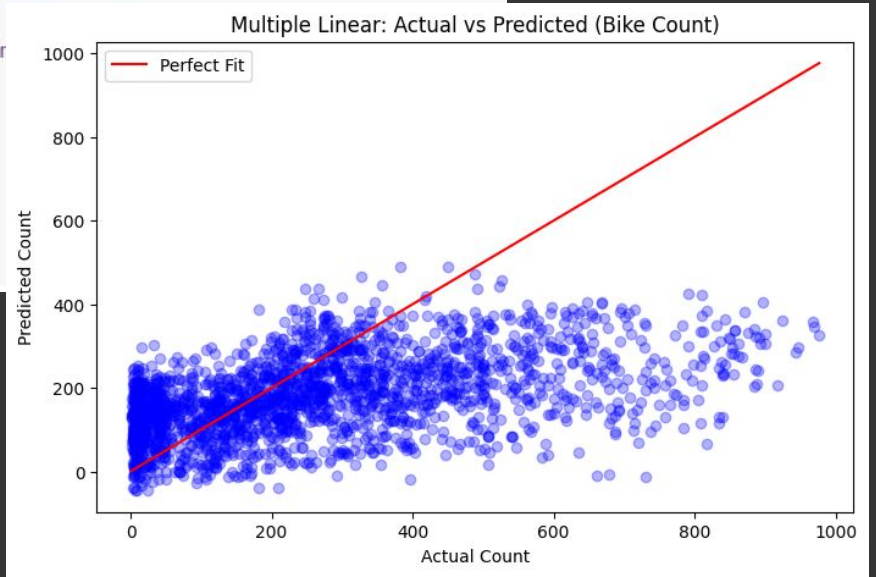
Multiple Linear Bikeshare

```python
model_multi = SGDRegressor(learning_rate='adaptive', eta0=0.001, max_iter=1, warm_start=True)
```

✓ [31] 34ms

```python
plt.figure(figsize=(8, 5))
plt.scatter(bikeshare_y_test, bikeshare_multi_pred, alpha=0.3, color='blue')
plt.plot([bikeshare_y_test.min(), bikeshare_y_test.max()],
         [bikeshare_y_test.min(), bikeshare_y_test.max()], color
plt.title('Multiple Linear: Actual vs Predicted (Bike Count)')
plt.xlabel('Actual Count')
plt.ylabel('Predicted Count')
plt.legend()
plt.show()
```

✓ [34] 200ms



Multiple Linear Bikeshare (Cont.)

```python
bikeshare_multi_pred = model_multi.predict(bikeshare_test_full_scaled)
bikeshare_multi_r2 = r2_score(bikeshare_y_test, bikeshare_multi_pred)
bikeshare_multi_mse = np.sqrt(mean_squared_error(bikeshare_y_test, bikeshare_multi_pred))
print(f"R-squared: {bikeshare_multi_r2:.3f} (bikeshare_multi_r2)")
print(f"RMSE: {bikeshare_multi_mse:.2f}")
```

✓ [35] 49ms

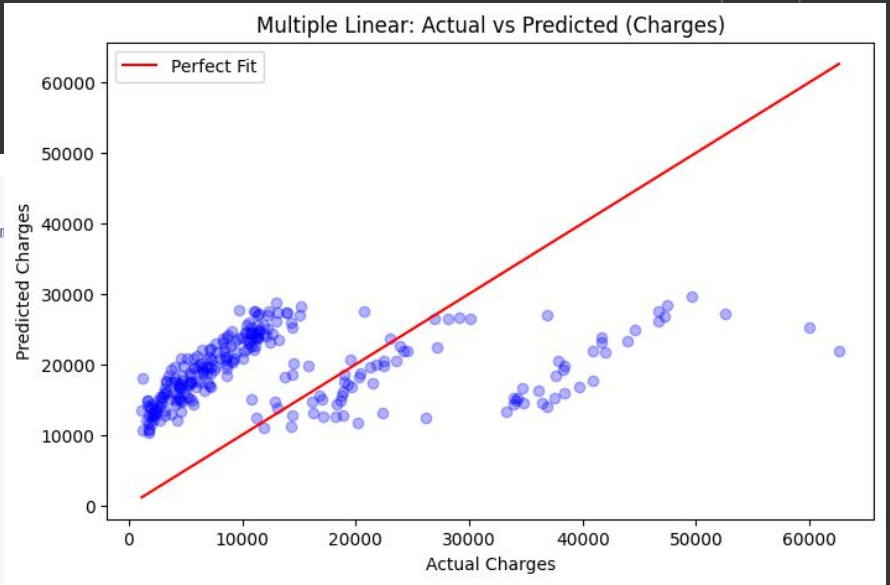
```
R-squared: 0.096 (bikeshare_multi_r2)
RMSE: 206.92
```

- 9% better than average!!
- Data may not map linearly
- May be noise introduced

Multiple Linear Insurance

```python
# Multiple Linear Regression - Insurance
model_multi = SGDRegressor(learning_rate='adaptive', eta0=0.0001, max_iter=1, warm

for i in range(0, insurance_train_record_count, insurance_train_chunk_size):
    model_multi.partial_fit(
        insurance_X_train_all[i:i + insurance_train_chunk_size],
        insurance_y_train[i:i + insurance_train_chunk_size]
    )

insurance_multi_pred = model_multi.predict(insurance_X_test_all)

plt.figure(figsize=(8, 5))
plt.scatter(insurance_y_test, insurance_multi_pred, alpha=0.3, color='blue')
plt.plot([insurance_y_test.min(), insurance_y_test.max()],
         [insurance_y_test.min(), insurance_y_test.max()], color='red', label='Perfect Fit')
plt.title('Multiple Linear: Actual vs Predicted (Charges)')
plt.xlabel('Actual Charges')
plt.ylabel('Predicted Charges')
plt.legend()
plt.show()
```

✓ [38] 215ms



Clustering!!

Multiple Linear Insurance (Cont.)

- Clustering is unsurprising
- https://dl.4d2.sh/AzaGfDsbyjq4.html

```
insurance_multi_r2 = r2_score(insurance_y_test, insurance_multi_pred)
insurance_multi_mse = np.sqrt(mean_squared_error(insurance_y_test, insurance_multi_pred))
print(f"R-squared: {insurance_multi_r2:.3f} (insurance_multi_r2)")
print(f"RMSE: {insurance_multi_mse:.2f}")
```

✓ [39] 41ms

```
R-squared: -0.132 (insurance_multi_r2)
RMSE: 13112.63
```